Correção do exercício 8

Os erros mais comuns nas funções

Erro 1

RETURNS errado

A função devolve texto, mas o RETURNS diz numeric. O tipo tem de bater com o RETURN.

Erro 2

Variável sem INTO

O SELECT busca o valor, mas sem INTO ele não vai para a variável.

Erro 3

IF sem END IF

Cada IF fecha com END IF;. Os ELSIF não pedem um END IF próprio.

Procedures

Banco de Dados 2 · Aula 19

SQL: Structured Query Language · PL/pgSQL: Procedural Language/PostgreSQL

NOVO ASSUNTO

Procedure

A função responde uma pergunta. Mas a loja também precisa que o banco faça coisas:

- Gravar o pagamento e mudar o pedido para pago, numa chamada só.
- Testar as regras antes de gravar, e parar se algo estiver errado.

Uma ação do negócio guardada no banco, com nome e parâmetros, é uma procedure.

Função ou procedure?

Pergunta ou ação

Função

Responde

Devolve um valor (RETURNS e RETURN). É chamada num SELECT: SELECT calcular_frete(...).

Procedure

Faz

Não devolve valor e muda dados. É chamada com CALL: CALL registrar_pagamento(...).

Registrar o pagamento de um pedido: é uma pergunta ou uma ação?

Onde e por que usar uma procedure

Três ações da Loja Virtual

Pagamento

Registrar o pagamento

Grava o pagamento e muda o status, juntos.

Estoque

Dar entrada no estoque

Soma a quantidade e registra o movimento.

Rotina

Cancelar os vencidos

Cancela de uma vez os pedidos não pagos no prazo.

registrar_pagamento (1 de 6)

OBJETIVO **O cliente pagou: grave o pagamento e passe o pedido para pago**

```
1 CREATE OR REPLACE PROCEDURE   -- não é função
2   registrar_pagamento(   -- o nome
3   p_pedido integer,   -- o pedido
4   p_forma varchar,   -- pix, cartao ou boleto
5   p_valor numeric)   -- o valor pago
6 LANGUAGE plpgsql AS $$   -- sem RETURNS
7 DECLARE p pedido%ROWTYPE;   -- a linha do pedido
```

Procedure não tem RETURNS: ela faz, não devolve. O resto é igual à função: DECLARE, BEGIN, SELECT ... INTO.

registrar_pagamento (2 de 6)

OBJETIVO **A regra 1: o pedido existe?**

```
1 BEGIN   -- começa o corpo
2   SELECT * INTO p FROM pedido   -- busca
3   WHERE id = p_pedido;   -- o pedido
4   IF NOT FOUND THEN   -- regra 1
5     RAISE EXCEPTION   -- para
6       'Pedido % não existe', p_pedido;   -- motivo
7   END IF;   -- fim
```

Cada regra é um IF com RAISE EXCEPTION, antes de gravar qualquer coisa.

registrar_pagamento (3 de 6)

OBJETIVO **A regra 2: o pedido espera o pagamento?**

```
1 IF p.status <> 'aguardando_pagamento'   -- regra 2
2 THEN RAISE EXCEPTION   -- para
3   'Pedido % está %',   -- motivo
4   p_pedido, p.status;   -- os dois %
5 END IF;   -- fim
```

<> quer dizer «diferente de». O primeiro % recebe o pedido; o segundo, o status: «Pedido 2 está pago».

registrar_pagamento (4 de 6)

OBJETIVO **A regra 3: o valor é o total do pedido?**

```
IF p_valor <> p.total THEN   -- regra 3
  RAISE EXCEPTION   -- para
    'Valor % diferente do total (%)',   -- motivo
    p_valor, p.total;   -- os dois %
END IF;   -- fim
```

O valor pago tem de ser igual ao total do pedido.

registrar_pagamento (5 de 6)

OBJETIVO **A regra 4: o prazo de 48 horas passou?**

```
1 IF p.fechado_em + INTERVAL '48 hours'   -- regra 4
2    < LOCALTIMESTAMP THEN   -- passou?
3    RAISE EXCEPTION   -- para
4      'Prazo vencido para o pedido %',   -- motivo
5      p_pedido;   -- o %
6 END IF;   -- fim
```

INTERVAL e LOCALTIMESTAMP são da aula de funções de data: a hora em que o pedido foi fechado, mais 48 horas, comparada com agora.

registrar_pagamento (6 de 6)

OBJETIVO **Passou por todas as regras: agora, a ação**

```
    INSERT INTO pagamento   -- 1ª ação: grava
      (pedido_id, forma, valor)   -- as colunas
    VALUES (p_pedido, p_forma, p_valor);   -- valores
    UPDATE pedido SET status = 'pago'   -- 2ª ação
    WHERE id = p_pedido;   -- só este pedido
END; $$;   -- fim
```

Primeiro as regras, depois a ação. Se uma regra falha, o RAISE EXCEPTION para tudo antes do INSERT.

Chamar com CALL e conferir

OBJETIVO **O pagamento do pedido 61 foi gravado?**

```sql
SELECT status FROM pedido WHERE id = 61;   -- antes
CALL registrar_pagamento(   -- chama
  61, 'pix', 1002.60);   -- pedido, forma, valor
SELECT status FROM pedido WHERE id = 61;   -- agora
SELECT COUNT(*) FROM pagamento;   -- 90 vira 91
```

A procedure não mostra nada: quem mostra é o SELECT seguinte. Antes: aguardando_pagamento. Depois: pago.

Os testes que falham

OBJETIVO **Qual mensagem cada chamada mostra?**

```
1 CALL registrar_pagamento(2,   -- pedido 2
2   'pix', 1295.40);   -- erro: está pago
3 CALL registrar_pagamento(73,   -- pedido 73
4   'pix', 500);   -- erro: o valor
5 CALL registrar_pagamento(1,   -- pedido 1
6   'pix', 1002.60);   -- erro: o prazo
7 SELECT COUNT(*) FROM pagamento;   -- continua 91
```

Rode uma chamada de cada vez e preveja a mensagem antes. Depois de cada erro, o pedido continua como estava.

O erro desfaz tudo

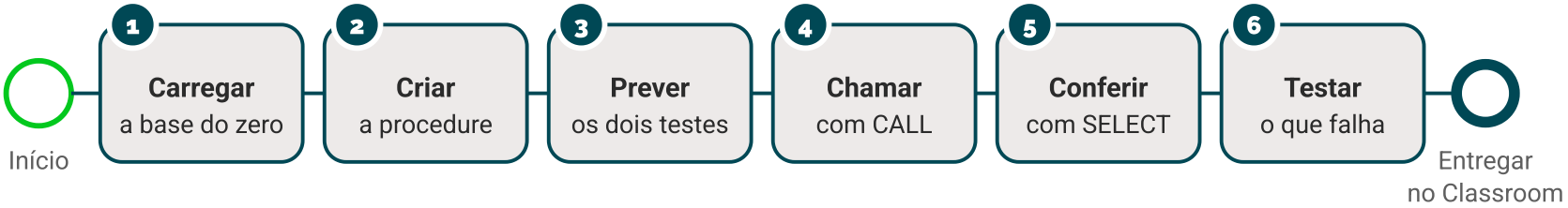
OBJETIVO O que acontece com o INSERT se a procedure para depois dele?

```
1 CREATE OR REPLACE PROCEDURE   -- uma procedure
2   testar_desfazer()   -- de teste
3 LANGUAGE plpgsql AS $$ BEGIN   -- corpo
4   INSERT INTO movimento_estoque   -- 1º: grava
5     (produto_id, tipo, quantidade)   -- colunas
6     VALUES (1, 'ajuste', 1);   -- um ajuste
7   RAISE EXCEPTION 'Parei no meio';   -- 2º: para
8 END; $$;   -- fim
```

Conte os ajustes antes e depois de CALL testar_desfazer();: o número não muda.

Exercício 9: procedures

Pode fazer em dupla. Cada aluno entrega o seu, no Classroom



Faça as 7 procedures, uma de cada vez. Elas mudam os dados: para rodar o arquivo de novo, carregue a base do zero.

Para fixar: responda de memória

Sem olhar o material e sem nota. Escreva as respostas em comentários, no fim do arquivo

Pergunta 1

Função ou procedure?

Qual devolve um valor?
Como se chama cada uma?

Pergunta 2

Conferir

Depois de um CALL, como você confere o que a procedure fez?

Pergunta 3

O erro

O que acontece com os dados se a procedure para no meio?